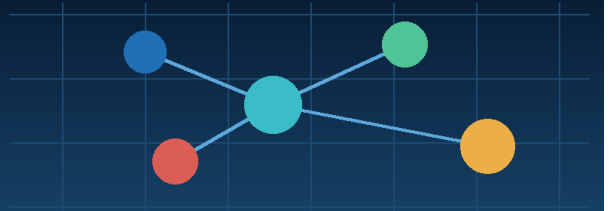


CORE PLATFORM FRAMEWORK

OFFICIAL PRODUCT DOCUMENTATION

Build · Run · Recover · Audit

master 64fd08d96392 | source 4c4248a12e69 | 186 requirements



CORE PLATFORM FRAMEWORK

01 개발자 매뉴얼

Generator → API → Domain → DB → Transaction → Integration → Security → Test → 운영
인계

항목	기준
대상	Java/Spring 기반 온라인·연계·Frontend 개발자, Tech Lead
정보 역할	온라인 개발과 Public API/SPI 사용 정보
기준 Repository	freeangelsun/202412_01_CPF · master
기준 Commit	64fd08d963927860e8d023403dfa276931801ee5
요구사항 정보	CPF_FINAL_TARGET_REQUIREMENTS.md · 186 개
현행화 Source	4c4248a12e699c07f9f5fb11fbb33b97ca04077d

이 문서는 현재 구현의 상태 보고가 아니라, CPF 제품 사용자가 업무를 끝내기 위해 따라야 할 완성 기준 사용 계약을 중심으로 작성한다. 실제 CPF Symbol/Path 는 CURRENT SOURCE, 구현 독립 예시는 REFERENCE 로 구분한다.

문서 읽는 법과 사실 구분

표기	의미	사용 규칙
CURRENT SOURCE	최신 master에서 직접 확인한 실제 Class/API/Route/Property/SQL/Test/경로	복사·검색 가능한 실제 이름으로 사용한다.
PRODUCT CONTRACT	CPF가 완료된 제품으로서 제공해야 하는 사용·운영 계약	현재 구현량 때문에 축소하지 않는다.
REFERENCE	사용자가 그대로 응용할 수 있도록 제공하는 완성 기준 코드·설정·절차 예시	실제 CPF 내부 Symbol인 것처럼 표현하지 않는다.

본문은 기능을 찾는 데서 끝나지 않는다. 적용 가능한 범위에서 선택 기준 → 준비 → 구현/설정 → 실행 → 정상 결과 → 실패/부분 실패 → 복구·대사 → 보안·감사 → Test → 운영 인계 순서로 설명한다.

1. 개발 환경과 Build

[CURRENT SOURCE] Repository 개발 환경

항목	기준/명령
JDK	25
Build	./gradlew clean build
Backend Test	./gradlew test
Quality	./gradlew qualityGate
Frontend	npm ci → lint → typecheck → test → build
Artifact Mode	LOCAL_DEV / REMOTE / OFFLINE

Linux/macOS

```
● ● ● TERMINAL · Linux/macOS

java -version
./gradlew --version
./gradlew clean build
./gradlew test
./gradlew qualityGate
```

Windows PowerShell

```
● ● ● TERMINAL · Windows PowerShell

java -version
.\gradlew.bat --version
.\gradlew.bat clean build
.\gradlew.bat test
.\gradlew.bat qualityGate
```

1.1 Artifact 공급 모드

모드	입력	실패 정책
LOCAL_DEV	~/.cpf/repository 또는 CPF_LOCAL_ARTIFACT_REPOSITORY	검증된 local repository 만 사용
REMOTE	CPF_ARTIFACT_REPOSITORY_URL + credential	Remote 실패를 Local 로 조용히 fallback 하지 않음
OFFLINE	CPF_OFFLINE_ARTIFACT_REPOSITORY	manifest/checksum 이 있는 bundle 만 사용

2. Profile/Starter 선택과 Generator

[CURRENT SOURCE] settings.gradle / cpf-starter-catalog.json

Profile	권장 시작점
minimal-domain	순수 Domain/contract
web-api	REST 업무 API
secure-api	보호된 API
browser-bff	Browser session/BFF
event-service	Event/Broker
batch-service	Batch 업무

Generator 입력 예

● ● ● CODE · 신규 Domain 생성 입력 예

```
# REFERENCE - 신규 Domain 생성 입력 예
DomainName=Payment
SystemCode=PAY
Package=com.customer.payment
DatabaseVendor=postgresql
Profile=secure-api
Capabilities=data,messaging,integration,security
```

2.1 생성 결과를 검수하는 순서

1. Manifest 의 resolvedStarters 와 profile version 확인
2. build.gradle 이 CPF BOM/Convention Plugin 사용
3. API/Application/Domain/Persistence/Test/Migration 폴더 확인
4. DB Vendor pack 과 schema owner 확인
5. OpenAPI/JavaDoc/Test 가 같은 변경 단위인지 확인
6. 사용자 소유 영역과 Generator-owned 영역 경계 확인

3. 신규 업무 기능 전체 Reference - Transfer

[REFERENCE] 완성 기준 업무 예제. 실제 CPF 내부 업무 Symbol 이 아니라 고객 Domain 구현 패턴이다.

이 절은 개발자가 CPF 를 처음 사용해도 하나의 Command 업무를 끝까지 구현할 수 있도록

API→Application→Domain→Persistence→Transaction→Idempotency→외부 호출→Event→Test 까지 연결한다. 아래 패키지명은 고객 예시다.

Reference 파일 구조

● ● ● TREE · TransferFacade.java

```
com.customer.payment.transfer/  
├─ api/  
│   ├── TransferFacade.java  
│   ├── TransferRequest.java  
│   └── TransferResult.java  
├─ application/TransferService.java  
├─ domain/Transfer.java  
├─ port/AccountPort.java  
├─ adapter/remote/CpfAccountAdapter.java  
├─ adapter/persistence/TransferJdbcRepository.java  
├─ config/TransferConfiguration.java  
└─ test/TransferServiceTest.java
```

3.1 Public API 와 DTO

src/main/java/com/customer/payment/transfer/api/*

● ● ● SOURCE · src/main/java/com/customer/payment/transfer/api/*src/main/java/c

```
package com.customer.payment.transfer.api;  
  
import java.math.BigDecimal;  
import java.util.Objects;  
  
public record TransferRequest(  
    String requestId,  
    String idempotencyKey,  
    String debitAccountId,  
    String creditAccountId,  
    BigDecimal amount,  
    String currency) {  
    public TransferRequest {  
        if (requestId == null || requestId.isBlank()) throw new IllegalArgumentException("requestId");  
        if (idempotencyKey == null || idempotencyKey.isBlank()) throw new IllegalArgumentException("idempotencyKey");  
        if (debitAccountId == null || debitAccountId.isBlank()) throw new IllegalArgumentException("debitAccountId");  
        if (creditAccountId == null || creditAccountId.isBlank()) throw new IllegalArgumentException("creditAccountId");  
        if (Objects.equals(debitAccountId, creditAccountId)) throw new IllegalArgumentException("same account");  
        if (amount == null || amount.signum() <= 0) throw new IllegalArgumentException("amount");  
        if (currency == null || !currency.matches("[A-Z]{3}")) throw new IllegalArgumentException("currency");  
    }  
}  
  
public record TransferResult(String transferId, String state, String transactionId) {}  
  
public interface TransferFacade {  
    TransferResult transfer(TransferRequest request);  
}
```

Public DTO 는 transport/framework 구현 type 을 노출하지 않는다. requestId 는 호출 상관관계, idempotencyKey 는 업무상 동일 요청의 중복 제어에 사용한다.

3.2 Domain 상태와 Application Service

src/main/java/com/customer/payment/transfer/application/TransferService.java

● ● ● SOURCE · src/main/java/com/customer/payment/transfer/application/TransferService.java

```
package com.customer.payment.transfer.application;

import com.customer.payment.transfer.api.*;
import com.customer.payment.transfer.port.AccountPort;
import java.time.Clock;
import java.util.UUID;

public final class TransferService implements TransferFacade {
    private final TransferRepository repository;
    private final AccountPort accountPort;
    private final TransferEventPort eventPort;
    private final Clock clock;

    public TransferService(TransferRepository repository, AccountPort accountPort,
        TransferEventPort eventPort, Clock clock) {
        this.repository = repository;
        this.accountPort = accountPort;
        this.eventPort = eventPort;
        this.clock = clock;
    }

    @Override
    public TransferResult transfer(TransferRequest r) {
        var replay = repository.findByIdempotencyKey(r.idempotencyKey());
        if (replay.isPresent()) return replay.get().toResult();

        var transfer = Transfer.pending(UUID.randomUUID().toString(), r, clock.instant());
        repository.insertPending(transfer);
        try {
            var external = accountPort.transfer(transfer.toAccountCommand());
            transfer.markExternalAccepted(external.providerRequestId());
            repository.commitAccepted(transfer);
            eventPort.enqueueCompleted(transfer.toEvent());
            return transfer.toResult();
        } catch (RemoteResultUnknownException e) {
            transfer.markUnknown(e.providerRequestId());
            repository.markUnknown(transfer);
            return transfer.toResult();
        } catch (RuntimeException e) {
            repository.rollbackPending(transfer.id());
            throw e;
        }
    }
}
```

외부 호출 성공 후 로컬 commit 실패 가능성이 있다면 Local rollback 이 외부 side effect 를 되돌린다고 가정하지 않는다. UNKNOWN_RESULT 로 보존하고 결과 조회/Reconcile 경로를 둔다.

3.3 Migration 과 Repository

db/migration/V001__create_pay_transfer.sql

● ● ● SQL · db/migration/V001__create_pay_transfer.sql

```
CREATE TABLE pay_transfer (
  transfer_id   VARCHAR(64) NOT NULL,
  idempotency_key VARCHAR(128) NOT NULL,
  request_hash  CHAR(64) NOT NULL,
  state         VARCHAR(24) NOT NULL,
  provider_request_id VARCHAR(128),
  version_no    BIGINT NOT NULL DEFAULT 0,
  created_at    TIMESTAMP NOT NULL,
  updated_at    TIMESTAMP NOT NULL,
  PRIMARY KEY (transfer_id),
  CONSTRAINT uq_pay_transfer_idem UNIQUE (idempotency_key)
);
CREATE INDEX ix_pay_transfer_state_time ON pay_transfer(state, updated_at);
```

src/main/java/com/customer/payment/transfer/adapter/persistence/TransferJdbcRepository.java

● ● ● SQL · src/main/java/com/customer/payment/transfer/adapter/persistence/TransferJdbcRepository.java

```
public final class TransferJdbcRepository implements TransferRepository {
  private final JdbcTemplate jdbc;

  public Optional<Transfer> findByIdempotencyKey(String key) {
    return jdbc.query("SELECT * FROM pay_transfer WHERE idempotency_key = ?", mapper, key)
      .stream().findFirst();
  }

  public void markUnknown(Transfer t) {
    int n = jdbc.update("""
      UPDATE pay_transfer SET state='UNKNOWN', provider_request_id=?,
      version_no=version_no+1, updated_at=CURRENT_TIMESTAMP
      WHERE transfer_id=? AND version_no=?
      """, t.providerRequestId(), t.id(), t.version());
    if (n != 1) throw new OptimisticLockException(t.id());
  }
}
```

3.4 REST 요청/응답

정상 요청

● ● ● OUTPUT · 정상 요청

```
POST /api/transfers
Content-Type: application/json
Idempotency-Key: pay-20260808-000001

{
  "requestId": "REQ-20260808-000001",
  "idempotencyKey": "pay-20260808-000001",
  "debitAccountId": "A1001",
  "creditAccountId": "A2001",
  "amount": 125000.00,
  "currency": "KRW"
}
```

정상 응답

● ● ● OUTPUT · 정상 응답

HTTP/1.1 202 Accepted
Content-Type: application/json

```
{  
  "transferId": "TR-01J...",  
  "state": "ACCEPTED",  
  "transactionId": "<34-char trusted transaction id>"  
}
```

중복 키 + 다른 request hash

● ● ● HTTP · 중복 키 + 다른 request hash

HTTP/1.1 409 Conflict

```
{  
  "code": "PAY-TRANSFER-IDEMPOTENCY-CONFLICT",  
  "message": "동일 idempotency key 의 요청 내용이 다릅니다.",  
  "retryable": false,  
  "transactionId": "..."  
}
```

4. Paging - 실제 CPF Public API 사용

[CURRENT SOURCE] cpf-core/src/main/java/com/cpf/core/api/page/CpfPageRequest.java

CURRENT SOURCE excerpt: CpfPageRequest.java

● ● ● SOURCE · CpfPageRequest.java

```
public record CpfPageRequest(int page, int size) {
    public static final int DEFAULT_SIZE = 20;
    public static final int MAX_SIZE = 200;

    public CpfPageRequest {
        if (page < 0) throw new IllegalArgumentException("page 는 0 이상이어야 합니다.");
        if (size < 1 || size > MAX_SIZE)
            throw new IllegalArgumentException("size 는 1~" + MAX_SIZE + " 범위여야 합니다.");
    }

    public static CpfPageRequest of(Integer page, Integer size) {
        return new CpfPageRequest(page == null ? 0 : page,
            size == null ? DEFAULT_SIZE : size);
    }

    public int offset() { return Math.multiplyExact(page, size); }
}
```

4.1 Repository sort allow-list

REFERENCE - 안전한 정렬 매핑

● ● ● SOURCE · REFERENCE - 안전한 정렬 매핑

```
private static final Map<String,String> SORT_COLUMNS = Map.of(
    "createdAt", "created_at",
    "status", "status",
    "businessKey", "business_key"
);

String column = SORT_COLUMNS.get(request.sort().field());
if (column == null) throw new IllegalArgumentException("unsupported sort field");
String order = request.sort().direction() == ASC ? "ASC" : "DESC";
// column/order are allow-listed tokens; user input is never concatenated directly.
String sql = "SELECT ... ORDER BY " + column + " " + order + " LIMIT ? OFFSET ?";
```

선택	사용 기준	주의
Offset Page	관리 화면/중소 결과, total count 필요	deep page 비용
Slice	다음 페이지 존재 여부만 필요	size+1 lookahead
Cursor	대용량/변경 많은 데이터	stable sort key, signed/versioned/expiry cursor

5. Transaction 전략별 개발법

5.1 LOCAL

항목	내용
목적	LOCAL 방식으로 업무 일관성을 구현
대상 역할	아키텍트/개발자
Owner	cpf-core 전역 semantics + Owner 별 transaction/data capability contract + selected provider starter/business owner
실제/대표 Consumer	단일 DB Aggregate 변경
선행 조건	Provider/DB/operation identity 준비
입력·기본값·범위	단일 DB Aggregate 변경
정상 흐름	@Transactional/TransactionTemplate, 짧은 경계, 외부 네트워크를 긴 TX 안에 두지 않음
정상 판정	LOCAL terminal state 와 업무 원장 일치
오류·경계·부분 실패	DB 예외→rollback; commit 전/후 장애 구분
복구·대사·Rollback	결과 조회→idempotent retry/recovery→필요 시 compensation/manual review
권한·보안·감사	민감정보 masking, 관리자 강제조치 Reason/Approval/Audit
Log·Metric·Trace	transactionId/segment/attempt, state/count/latency
Test	정상+timeout+duplicate+process kill+recovery
운영 인계	ADM Timeline/Recovery 에서 추적

5.2 XA/JTA

항목	내용
목적	XA/JTA 방식으로 업무 일관성을 구현
대상 역할	아키텍트/개발자
Owner	cpf-core 전역 semantics + Owner 별 transaction/data capability contract + selected provider starter/business owner
실제/대표 Consumer	DB+DB/DB+JMS 강한 원자성
선행 조건	Provider/DB/operation identity 준비
입력·기본값·범위	DB+DB/DB+JMS 강한 원자성
정상 흐름	Provider-neutral TM + XA resources, 2PC timeout/recovery log
정상 판정	XA/JTA terminal state 와 업무 원장 일치
오류·경계·부분 실패	prepare 이후 kill→startup recovery scan, heuristic/in-doubt 운영 경로
복구·대사·Rollback	결과 조회→idempotent retry/recovery→필요 시 compensation/manual review

항목	내용
권한·보안·감사	민감정보 masking, 관리자 강제조치 Reason/Approval/Audit
Log·Metric·Trace	transactionId/segment/attempt, state/count/latency
Test	정상+timeout+duplicate+process kill+recovery
운영 인계	ADM Timeline/Recovery 에서 추적

XA/JTA Reference + crash recovery harness

● ● ● SOURCE · XA/JTA Reference + crash recovery harnessXA/JTA Reference + cras

```

// REFERENCE - provider-neutral boundary
public interface XaTransactionCoordinator {
    XaOutcome execute(Duration timeout, List<XaParticipant> participants, Runnable businessWork);
    RecoveryReport recoverInDoubt(int limit);
}

// Harness flow
begin();
enlist(databaseXaResource);
enlist(jmsXaResource);
prepareAll();
killProcessAfterPrepare(); // fault injection point
restartTransactionManager();
RecoveryReport report = coordinator.recoverInDoubt(100);
assertNoDuplicateBusinessSideEffect();

```

5.3 OUTBOX

항목	내용
목적	OUTBOX 방식으로 업무 일관성을 구현
대상 역할	아키텍트/개발자
Owner	cpf-core 전역 semantics + Owner 별 transaction/data capability contract + selected provider starter/business owner
실제/대표 Consumer	DB+Event
선행 조건	Provider/DB/operation identity 준비
입력·기본값·범위	DB+Event
정상 흐름	Business update + Outbox insert 동일 Local TX
정상 판정	OUTBOX terminal state 와 업무 원장 일치
오류·경계·부분 실패	Publisher ACK loss, lease/fence, duplicate-safe publish
복구·대사·Rollback	결과 조회→idempotent retry/recovery→필요 시 compensation/manual review
권한·보안·감사	민감정보 masking, 관리자 강제조치 Reason/Approval/Audit
Log·Metric·Trace	transactionId/segment/attempt, state/count/latency
Test	정상+timeout+duplicate+process kill+recovery
운영 인계	ADM Timeline/Recovery 에서 추적

OUTBOX Local TX pattern

● ● ● SQL · OUTBOX Local TX pattern

```
BEGIN;  
UPDATE pay_transfer SET state='ACCEPTED' WHERE transfer_id=?;  
INSERT INTO cpf_outbox(event_id, transaction_id, state, payload_hash, available_at)  
VALUES (?, ?, 'PENDING', ?, CURRENT_TIMESTAMP);  
COMMIT;
```

-- Publisher claims with lease/fence, publishes, then CONFIRMED.

-- ACK loss is UNKNOWN until provider/result reconciliation.

5.4 INBOX_DEDUP

항목	내용
목적	INBOX_DEDUP 방식으로 업무 일관성을 구현
대상 역할	아키텍트/개발자
Owner	cpf-core 전역 semantics + Owner 별 transaction/data capability contract + selected provider starter/business owner
실제/대표 Consumer	at-least-once consume
선행 조건	Provider/DB/operation identity 준비
입력 · 기본값 · 범위	at-least-once consume
정상 흐름	messageId+consumer identity 원장
정상 판정	INBOX_DEDUP terminal state 와 업무 원장 일치
오류 · 경계 · 부분 실패	동시 duplicate/kill/replay 에서 업무 side effect 1 회
복구 · 대사 · Rollback	결과 조회→idempotent retry/recovery→필요 시 compensation/manual review
권한 · 보안 · 감사	민감정보 masking, 관리자 강제조치 Reason/Approval/Audit
Log · Metric · Trace	transactionId/segment/attempt, state/count/latency
Test	정상+timeout+duplicate+process kill+recovery
운영 인계	ADM Timeline/Recovery 에서 추적

5.5 SAGA

항목	내용
목적	SAGA 방식으로 업무 일관성을 구현
대상 역할	아키텍트/개발자
Owner	cpf-core 전역 semantics + Owner 별 transaction/data capability contract + selected provider starter/business owner
실제/대표 Consumer	A→B→C 장기 workflow
선행 조건	Provider/DB/operation identity 준비
입력 · 기본값 · 범위	A→B→C 장기 workflow

항목	내용
정상 흐름	durable saga state + compensation metadata
정상 판정	SAGA terminal state 와 업무 원장 일치
오류 · 경계 · 부분 실패	compensation 자체 실패/UNKNOWN/MANUAL_REVIEW
복구 · 대사 · Rollback	결과 조회→idempotent retry/recovery→필요 시 compensation/manual review
권한 · 보안 · 감사	민감정보 masking, 관리자 강제조치 Reason/Approval/Audit
Log · Metric · Trace	transactionId/segment/attempt, state/count/latency
Test	정상+timeout+duplicate+process kill+recovery
운영 인계	ADM Timeline/Recovery 에서 추적

Saga 상태

● ● ● FLOW · Saga 상태

STARTED → RUNNING → COMPLETED
 ↳ FAILED → COMPENSATING → COMPENSATED
 ↳ UNKNOWN → MANUAL_REVIEW

5.6 TCC

항목	내용
목적	TCC 방식으로 업무 일관성을 구현
대상 역할	아키텍트/개발자
Owner	cpf-core 전역 semantics + Owner 별 transaction/data capability contract + selected provider starter/business owner
실제/대표 Consumer	Hold/Reservation
선행 조건	Provider/DB/operation identity 준비
입력 · 기본값 · 범위	Hold/Reservation
정상 흐름	Try/Confirm/Cancel contract
정상 판정	TCC terminal state 와 업무 원장 일치
오류 · 경계 · 부분 실패	empty rollback/hanging/duplicate confirm/cancel
복구 · 대사 · Rollback	결과 조회→idempotent retry/recovery→필요 시 compensation/manual review
권한 · 보안 · 감사	민감정보 masking, 관리자 강제조치 Reason/Approval/Audit
Log · Metric · Trace	transactionId/segment/attempt, state/count/latency
Test	정상+timeout+duplicate+process kill+recovery
운영 인계	ADM Timeline/Recovery 에서 추적

TCC 공개 의미

● ● ● CODE · TCC 공개 의미

```
Try(requestId, resource, ttl) // reserve, idempotent  
Confirm(requestId, reservationId) // commit reserve, idempotent  
Cancel(requestId, reservationId) // release reserve, empty-rollback safe
```

6. 실제 Online A→B→C(/D) Reference

[CURRENT SOURCE] [cpf-reference/online/integrated](#)

현재 master 에는 Controller→Domain A→B→C(/D)→Repository/Remote 의 실행형 Reference 가 있으며, transactionId 는 retry 에서도 유지되고 attempt 만 증가한다. 외부 side effect 성공 뒤 로컬 저장 실패는 UNKNOWN 으로 보존하고 D reconcile 에서 compensation 한다.

CURRENT SOURCE 핵심: `OnlineAbcdReferenceFlow.java`

● ● ● SOURCE · `OnlineAbcdReferenceFlow.java`

```
public enum Outcome { SUCCESS, FAILED, CONFLICT, UNKNOWN, RECONCILED }

public record Request(String transactionId, String businessKey,
    String payload, int attempt) {}

// Domain B: local transaction + Domain C remote side effect
String remote = c.execute(r, events);
repo.save(r.businessKey(), remote);
repo.commit(r.businessKey());

// local DB failure after remote success
boolean remoteSucceeded = events.stream().anyMatch(
    x -> x.segment().equals("C") && x.state().equals("REMOTE_SUCCESS"));
return new Result(r.transactionId(), r.businessKey(), r.attempt(),
    remoteSucceeded ? Outcome.UNKNOWN : Outcome.FAILED, List.copyOf(events));
```

CURRENT SOURCE: `OnlineAbcdReferenceHttpController.java`

● ● ● SOURCE · `OnlineAbcdReferenceHttpController.java`

```
@RestController
@Profile({"cpf-reference-online-abcd", "cpf-reference-online-abcd-jdbc"})
@RequestMapping("/reference/online/abcd")
public final class OnlineAbcdReferenceHttpController {
    @PostMapping
    public Result execute(@RequestBody Request request) {
        var tx = transactional.getIfAvailable();
        return tx == null ? controller.execute(request) : tx.execute(request);
    }
}
```

CURRENT SOURCE: `OnlineAbcdSpringTransactionService.java`

● ● ● FLOW · `OnlineAbcdSpringTransactionService.java`

```
this.required.setPropagationBehavior(TransactionDefinition.PROPROPAGATION_REQUIRED);
Result result = required.execute(status -> {
    Result current = controller.execute(request);
    if (current.outcome() == FAILED || current.outcome() == UNKNOWN || current.outcome() == CONFLICT)
        status.setRollbackOnly();
    return current;
});
```

6.1 실제 Test 가 검증하는 항목

- 성공 후 동일 transactionId retry 는 저장된 결과 재사용
- 다른 transactionId 의 같은 business key 는 CONFLICT

- A/B/C/remote failure 는 local rollback
- Remote 성공 후 DB save 실패는 UNKNOWN → D reconcile
- Remote timeout 후 같은 transactionId + attempt 증가 retry
- D reconcile 실패는 UNKNOWN 유지
- 동시 동일 business key 는 remote side effect 1 회
- 기존 DB 값이 있는 상태에서 실패하면 rollback 으로 기존 값 복원

7. Idempotency 와 동시성

Idempotency 는 “같은 key 면 성공”이 아니라 scope + canonical request hash + 상태 + 결과 + TTL 의 계약이다. 같은 key 라도 request hash 가 다르면 conflict 다. 처리 중인 key 에 동시 요청이 오면 wait/reject/poll 중 정책을 명확히 한다.

REFERENCE - 멍등 원장

● ● ● SOURCE · REFERENCE - 멍등 원장

```
record IdempotencyRecord(  
    String scope,  
    String key,  
    String requestHash,  
    State state, // PROCESSING / COMPLETED / FAILED / UNKNOWN  
    String resultRef,  
    Instant expiresAt,  
    long version) {}
```

7.1 실제 CpfLockManager

[CURRENT SOURCE] cpf-core/src/main/java/com/cpf/core/api/locking/CpfLockManager.java

Operation	중요 상태
acquire	ACQUIRED / IDEMPOTENT_REPLAY / BUSY / RESOURCE_EXHAUSTED / UNKNOWN / INVALID
renew	RENEWED / STALE_TOKEN / NOT_OWNER / NOT_FOUND / EXPIRED / UNKNOWN / INVALID
release	RELEASED / IDEMPOTENT_REPLAY / STALE_TOKEN / NOT_OWNER / NOT_FOUND / EXPIRED / UNKNOWN / INVALID
forceRelease	Reason + Approval + expected fence/epoch/version 을 바인딩

REFERENCE - lease/fencing 사용 패턴

● ● ● CODE · REFERENCE - lease/fencing 사용 패턴

```
AcquireResult acquired = lockManager.acquire(  
    "settlement:2026-08-08", instanceId, requestId, Duration.ofSeconds(30));  
if (acquired.status() != AcquireStatus.ACQUIRED  
    && acquired.status() != AcquireStatus.IDEMPOTENT_REPLAY) {  
    throw new BusyException(acquired.reason());  
}  
LockToken token = acquired.token();  
try {  
    if (!lockManager.validateToken(token)) throw new StaleWriterException();  
    // protected side effect  
} finally {  
    lockManager.release(token, "settlement completed");  
}
```

8. Remote call, Timeout Budget 와 UNKNOWN_RESULT

Deadline 분배 예

● ● ● TREE · Deadline 분배 예

```
request deadline 5s
├─ validation/auth 0.3s
├─ DB read        0.5s
├─ remote A       2.0s
├─ local commit   0.7s
└─ safety/serialization budget 1.5s
```

실패 시점	판정	다음 행동
연결 전 실패	FAILED / retryable 가능	역등성/정책 확인 후 bounded retry
요청 전송 중 단절	UNKNOWN 가능	상대 requestId 결과 조회
상대 side effect 성공 후 응답 유실	UNKNOWN_RESULT	재호출 전에 result query/reconcile
로컬 commit 실패 + remote 성공	UNKNOWN_RESULT	remote result + local ledger 대사 후 compensation/commit repair

9. Messaging - Kafka/JMS/IBM MQ/RabbitMQ

Provider	Starter 경계	개발자가 명시할 의미
Kafka	cpf-starter-messaging-kafka	key/partition/order, producer ACK, consumer offset
JMS 3.x	cpf-starter-messaging-jms	session/ack/redelivery/transaction
IBM MQ	cpf-starter-messaging-ibm-mq	Queue Manager/Channel/TLS/CCDT/connection recovery
RabbitMQ	cpf-starter-messaging-rabbitmq	exchange/routing/ack/nack/redelivery
Reliability JDBC	cpf-starter-messaging-reliability-jdbc	Outbox/Inbox/lease/fence/UNKNOWN/Reconcile

REFERENCE - Provider-neutral envelope 의미

● ● ● SOURCE · REFERENCE - Provider-neutral envelope 의미 REFERENCE - Provider-neu

```
record EventEnvelope(  
    String messageId,  
    String transactionId,  
    String schemaVersion,  
    String producer,  
    String environment,  
    String key,  
    Instant occurredAt,  
    Map<String,String> headers,  
    byte[] payload) {}
```

10. 외부 REST/TCP/전문/File 연계

HTTP

항목	내용
목적	HTTP 연계를 중복·오처리 없이 구현
대상 역할	Integration 개발자
Owner	cpf-core contract + selected integration/file starter
실제/대표 Consumer	업무 Adapter/External target
선행 조건	endpoint/layout/key/credential 준비
입력·기본값·범위	size/timeout/retry/auth
정상 흐름	Endpoint allowlist, auth, connect/response/overall timeout, retry safety, request/response masking, UNKNOWN result query.
정상 판정	provider ACK/response/checksum 과 업무 원장 일치
오류·경계·부분 실패	timeout/partial/connection loss/disk full/process kill
복구·대사·Rollback	결과 확인→safe retry/reconcile/cleanup
권한·보안·감사	SSRF/path traversal/secret/masking/audit
Log·Metric·Trace	integration log + txId + latency + status
Test	normal/negative/timeout/partial/recovery
운영 연계	ADM external/timeline 또는 owner operation

TCP

항목	내용
목적	TCP 연계를 중복·오처리 없이 구현
대상 역할	Integration 개발자
Owner	cpf-core contract + selected integration/file starter
실제/대표 Consumer	업무 Adapter/External target
선행 조건	endpoint/layout/key/credential 준비
입력·기본값·범위	size/timeout/retry/auth
정상 흐름	framing/encoding/heartbeat/reconnect/backoff/half-open/bounded queue/backpressure/correlation/TLS/response loss.
정상 판정	provider ACK/response/checksum 과 업무 원장 일치
오류·경계·부분 실패	timeout/partial/connection loss/disk full/process kill
복구·대사·Rollback	결과 확인→safe retry/reconcile/cleanup
권한·보안·감사	SSRF/path traversal/secret/masking/audit
Log·Metric·Trace	integration log + txId + latency + status

항목	내용
Test	normal/negative/timeout/partial/recovery
운영 인계	ADM external/timeline 또는 owner operation

Fixed Length - integration capability owner

항목	내용
목적	Fixed Length 연계를 중복·오처리 없이 구현
대상 역할	Integration 개발자
Owner	cpf-core contract + selected integration/file starter
실제/대표 Consumer	업무 Adapter/External target
선행 조건	endpoint/layout/key/credential 준비
입력·기본값·범위	size/timeout/retry/auth
정상 흐름	byte length, encoding, padding, field offset/group/version, streaming, masking. 기관별 layout/auth 는 고객 Adapter.
정상 판정	provider ACK/response/checksum 과 업무 원장 일치
오류·경계·부분 실패	timeout/partial/connection loss/disk full/process kill
복구·대사·Rollback	결과 확인→safe retry/reconcile/cleanup
권한·보안·감사	SSRF/path traversal/secret/masking/audit
Log·Metric·Trace	integration log + txId + latency + status
Test	normal/negative/timeout/partial/recovery
운영 인계	ADM external/timeline 또는 owner operation

File/SFTP

항목	내용
목적	File/SFTP 연계를 중복·오처리 없이 구현
대상 역할	Integration 개발자
Owner	cpf-core contract + selected integration/file starter
실제/대표 Consumer	업무 Adapter/External target
선행 조건	endpoint/layout/key/credential 준비
입력·기본값·범위	size/timeout/retry/auth
정상 흐름	bounded streaming, temp→fsync/checksum→atomic publish, path/symlink/archive protection, partial upload, cancellation, restart cleanup.
정상 판정	provider ACK/response/checksum 과 업무 원장 일치
오류·경계·부분 실패	timeout/partial/connection loss/disk full/process kill
복구·대사·Rollback	결과 확인→safe retry/reconcile/cleanup

항목	내용
권한·보안·감사	SSRF/path traversal/secret/masking/audit
Log·Metric·Trace	integration log + txId + latency + status
Test	normal/negative/timeout/partial/recovery
운영 인계	ADM external/timeline 또는 owner operation

11. Security 개발

OAuth2/JWT Developer API

currentUserId/currentTenantId/currentPrincipal/hasRole/hasScope/safe claim access, issuer/audience/expiry, token propagation, 401/403

OIDC SSO

Keycloak/Entra ID/Okta user/group/role/claim mapping, login/logout/session/token expiry/refresh, BFF

Secret/KMS/HSM

Provider health, keyId/version, rotation/revocation, PKCS#11 optional, no raw private key

Digital Signature

sign/verify, algorithm/keyId/keyVersion/certificate/signature metadata + audit

Tamper-evident Audit

canonical payload, previousHash/currentHash, ordering/concurrency/multi-instance, optional signature

AI Optional

provider/model routing, timeout/retry/fallback, masking, token/cost metering, authorization, high-risk approval

12. OpenAPI·JavaDoc·Generated Client

대상	필수 정보
Public Java API	입력/출력/nullability/thread-safety/failure/recovery/security/since/deprecated
REST OpenAPI	operationId, auth, header, request/response example, errors, idempotency, paging
Frontend generated client	exact-SHA spec, typed error/high-risk operation, drift gate
SPI	capability/lifecycle/version/failure/native escape hatch

13. Database Migration·Upgrade·Rollback

DB 변경은 Requirement/Data Model → Canonical Schema/Metadata → Generator/Golden Template → MariaDB/PostgreSQL/Oracle Vendor Source → Install/Migration/Rollback/Runtime Query → Consumer/Test → Fresh Runtime Evidence 순서로 관리한다.

Migration header 예

● ● ● CODE · Migration header 예

```
-- REFERENCE migration metadata
-- requirement: PAY-TRANSFER
-- owner: payment domain
-- expected lock: short metadata lock
-- data backfill: separate bounded step
-- rollback: restore previous column/index or forward-recovery plan
-- vendors: MariaDB / PostgreSQL / Oracle reviewed separately
```

14. Test 전략 - 정상만 통과시키지 않는다

Test	필수 범위
Unit	validation/domain policy/state/error mapping
Repository	migration, query/index, optimistic lock, rollback
Contract	Local/Remote parity, header/error/idempotency
Fault	timeout/response loss/dependency failure/disk full
Concurrency	duplicate race/lock/fencing/stale writer
Process	kill→restart→recovery
Security	401/403/CSRF/SSRF/injection/path/PII leak
Runtime	실제 DB/Broker/HTTP/multi-instance
Browser	login/search/detail/approval/download/retry/a11y

CURRENT SOURCE 의미를 따른 Fault Test 예

● ● ● CODE · CURRENT SOURCE 의미를 따른 Fault Test 예 CURRENT SOURCE 의미를 따른 Fault Te

```
@Test
void remote_success_then_local_failure_is_unknown_and_reconciled() {
    var f = fixture();
    f.repo.failSave = true;
    var unknown = f.controller.execute(req("TX-DB2", "KDB2", 1));
    assertEquals(Outcome.UNKNOWN, unknown.outcome());
    assertEquals(1, f.remote.sideEffects.get());

    var reconciled = f.d.reconcile(unknown);
    assertEquals(Outcome.RECONCILED, reconciled.outcome());
    assertEquals(0, f.remote.sideEffects.get());
}
```

15. EDU - 실행형 실습

15.1 Lab - LOCAL commit/rollback

항목	내용
목표	단일 DB commit 과 exception rollback
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.2 Lab - XA DB+DB

항목	내용
목표	두 XA resource 2PC
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.3 Lab - XA DB+JMS

항목	내용
목표	DB/JMS atomicity

항목	내용
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.4 Lab - XA prepare kill/recovery

항목	내용
목표	prepare 직후 process kill→restart scan
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.5 Lab - Outbox 동일 TX

항목	내용
목표	업무 update+outbox insert atomicity
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행

항목	내용
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.6 Lab - ACK loss/Publisher kill

항목	내용
목표	publish UNKNOWN→reconcile
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.7 Lab - Inbox/Dedup

항목	내용
목표	동시 duplicate business side effect 0
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행

항목	내용
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.8 Lab - Saga compensation

항목	내용
목표	C 실패→B/A 역순 compensation
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.9 Lab - Saga compensation failure

항목	내용
목표	retry/UNKNOWN/manual review
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.10 Lab - TCC confirm/cancel

항목	내용
목표	Try/Confirm/Cancel idempotency
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.11 Lab - TCC edge cases

항목	내용
목표	empty rollback/hanging/duplicate
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.12 Lab - External timeout

항목	내용
목표	UNKNOWN→result query→reconcile
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음

항목	내용
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.13 Lab - A→B→C Local/Remote

항목	내용
목표	같은 transactionId 와 parity
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.14 Lab - Batch→Domain

항목	내용
목표	batch execution identity + transaction lineage
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현

항목	내용
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.15 Lab - ADM Timeline

항목	내용
목표	한 transactionId 로 전체 segment 조회
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

15.16 Lab - Starter DX

항목	내용
목표	OSS direct 대비 boilerplate/fail-fast/escape hatch
선행 조건	clean profile, 필요한 DB/Broker/credential 은 test 전용으로 준비
Source 위치	cpf-reference 또는 고객 REFERENCE package. 실제 Product 내부 구현을 복제하지 않음
실행	해당 Test/Harness 를 clean 상태에서 실행
정상 결과	terminal state, transactionId/attempt, DB/Message side effect 수가 기대값과 일치
Fault Injection	timeout/kill/duplicate/partial 중 해당 시나리오를 재현
복구	result query/retry/reconcile/compensation/restart 중 계약에 맞는 경로 수행
ADM 확인	transactionId 로 Timeline/Unknown/Incident/Recovery 확인
Cleanup	test data, lock/lease, temp/spool 을 owner 절차로 정리
Evidence	명령, exit code, state before/after, artifact/log hash 를 기록

16. 배포 인계 체크리스트

- Artifact/BOM/Starter lock checksum
- DB migration/rollback/restore plan
- Secret/Certificate/Key owner 와 rotation
- OpenAPI/JavaDoc/current generated client
- Feature Flag/Config default 와 restart 여부
- SLO/Metric/Log/Trace dashboard
- Known failure→Runbook→Recovery owner
- ADM Permission/Approval/Audit
- Unit/Contract/Fault/Runtime/Browser Test evidence

Source Trace

구분	Repository 경로	확인 포인트
Build	settings.gradle	JDK/toolchain, Artifact mode, Profile/Starter physical closure
Starter Catalog	cpf-tools/generator/contracts/cpf-starter-catalog.json	Public profile, capability group, packageBase/configPrefix
Paging	cpf-core/src/main/java/com/cpf/core/api/page/CpfPageRequest.java	0-base/default 20/max 200/overflow safe
Lock	cpf-core/src/main/java/com/cpf/core/api/locking/CpfLockManager.java	lease/fence/owner epoch/version/approval
Online Reference	cpf-reference/src/main/java/com/cpf/reference/online/integrated/OnlineAbcdReferenceFlow.java	A→B→C(/D), UNKNOWN/Reconcile
HTTP Reference	cpf-reference/src/main/java/com/cpf/reference/online/integrated/OnlineAbcdReferenceHttpController.java	/reference/online/abcd
Spring TX	cpf-reference/src/main/java/com/cpf/reference/online/integrated/OnlineAbcdSpringTransactionService.java	REQUIRED + rollback-only
Fault Test	cpf-reference/src/test/java/com/cpf/reference/online/integrated/OnlineAbcdReferenceFlowTest.java	duplicate/concurrency/timeout/UNKNOWN/reconcile
EDU	cpf-docs/development/EDU_GUIDE.md	실행형 16 transaction reference track

17. 개발자가 바로 복사해 시작하는 기능별 Recipe

17.1 표준 CRUD

[PRODUCT CONTRACT]

항목	내용
구현 흐름	Controller→Facade→Application→Domain→Repository
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	validation/duplicate/optimistic lock
복구	DB rollback + 409 conflict
운영 확인	transactionId/operationId 로 ADM/owner query 확인

REFERENCE Controller

● ● ● SOURCE · REFERENCE Controller

```
@RestController
@RequestMapping("/api/v1/payees")
final class PayeeController {
    private final PayeeFacade facade;

    @PostMapping
    ResponseEntity<PayeeView> create(@Valid @RequestBody CreatePayeeRequest r,
        @RequestHeader("Idempotency-Key") String key) {
        return ResponseEntity.status(201).body(facade.create(r.withIdempotencyKey(key)));
    }

    @PutMapping("/{id}")
    PayeeView update(@PathVariable String id,
        @RequestHeader("If-Match") long version,
        @Valid @RequestBody UpdatePayeeRequest r) {
        return facade.update(id, version, r);
    }
}
```

REFERENCE concurrency test

● ● ● FLOW · REFERENCE concurrency test

```
@Test
void concurrent_update_returns_conflict_without_lost_update() throws Exception {
    Payee v1 = fixture.savedPayee();
    var f1 = pool.submit(() -> api.update(v1.id(), v1.version(), change("A")));
    var f2 = pool.submit(() -> api.update(v1.id(), v1.version(), change("B")));
    var results = List.of(capture(f1), capture(f2));
    assertEquals(1, results.stream().filter(Result::success).count());
    assertEquals(1, results.stream().filter(r -> r.status() == 409).count());
    assertEquals(2L, repository.find(v1.id()).version());
}
```

17.2 Offset/Slice Paging

[PRODUCT CONTRACT]

항목	내용
구현 흐름	CpfPageRequest + allowlisted sort
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	deep offset/invalid size/sort injection
복구	cursor 전환 기준
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.3 Cursor Paging

[PRODUCT CONTRACT]

항목	내용
구현 흐름	stable tuple + signed cursor
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	tamper/expiry/key rotation
복구	new cursor issuance
운영 확인	transactionId/operationId 로 ADM/owner query 확인

REFERENCE signed cursor

● ● ● SQL · REFERENCE signed cursor

```
record CursorPayload(Instant createdAt, String id, Instant expiresAt, int version) {}
String encode(CursorPayload p) {
    byte[] body = canonicalJson(p);
    byte[] sig = hmacSha256(activeKey(), body);
    return base64url(body) + "." + base64url(sig);
}
// decode: verify version/expiry/signature before using sort tuple
SELECT ... WHERE (created_at,id) < (?,?) ORDER BY created_at DESC,id DESC LIMIT ?;
```

17.4 Local Transaction

[PRODUCT CONTRACT]

항목	내용
구현 흐름	TransactionTemplate/@Transactional
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	deadlock/timeout/rollback
복구	bounded retry only if safe
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.5 Remote Call

[PRODUCT CONTRACT]

항목	내용
구현 흐름	Port + HTTP Adapter
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	connect/send/response loss
복구	UNKNOWN result query
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.6 Outbox Event

[PRODUCT CONTRACT]

항목	내용
구현 흐름	business TX + outbox row
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	ACK loss/publisher kill
복구	lease reclaim/reconcile
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.7 Inbox/Dedup

[PRODUCT CONTRACT]

항목	내용
구현 흐름	consumer identity + messageId
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	duplicate/race/kill
복구	resume/replay without duplicate effect
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.8 File Upload

[PRODUCT CONTRACT]

항목	내용
구현 흐름	bounded stream + temp/checksum/atomic publish
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	cancel/disk full/path attack
복구	cleanup/retry based on checksum
운영 확인	transactionId/operationId 로 ADM/owner query 확인

REFERENCE bounded upload

● ● ● CODE · REFERENCE bounded upload

```
Path temp = safeTemp(aliasRoot, requestId);
try (InputStream in = request.body(); OutputStream out = Files.newOutputStream(temp)) {
    long bytes = copyBoundedAndHash(in, out, maxBytes, digest);
    out.flush();
    fsync(temp);
}
verifyExpectedChecksum(temp);
atomicMove(temp, finalPath);
// on cancellation/disk failure: delete temp; never publish partial final target
```

17.9 SFTP Transfer

[PRODUCT CONTRACT]

항목	내용
구현 흐름	temp remote name + checksum/ack
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	partial upload/connection loss
복구	remote status/checksum
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.10 Fixed-Length - integration capability owner

[PRODUCT CONTRACT]

항목	내용
구현 흐름	layout/version/byte length/encoding
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	bad offset/encoding/oversize
복구	reject/quarantine
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.11 OAuth2/JWT

[PRODUCT CONTRACT]

항목	내용
구현 흐름	security context helper
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	issuer/audience/expiry/claim mismatch
복구	401/403 + token refresh outside resource server
운영 확인	transactionId/operationId 로 ADM/owner query 확인

REFERENCE developer security API

● ● ● CODE · REFERENCE developer security API

```
CpfPrincipal p = CpfSecurity.currentPrincipal();
String userId = p.requireUserId();
String tenant = p.requireTenantId();
if (!p.hasScope("payments.write")) throw CpfForbidden.scope("payments.write");
// raw token/claims Map 을 업무 전역에 전달하지 않는다.
```

17.12 OIDC BFF

[PRODUCT CONTRACT]

항목	내용
구현 흐름	server session + CSRF
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	session fixation/IdP down/refresh expiry
복구	reauth/session revoke
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.13 KMS/HSM Sign

[PRODUCT CONTRACT]

항목	내용
구현 흐름	keyId/version + sign/verify
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	provider timeout/revoked key
복구	retry/readiness/rotation policy
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.14 Tamper Audit

[PRODUCT CONTRACT]

항목	내용
구현 흐름	canonical payload + hash chain
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	concurrent sequence/hash break
복구	incident + chain verification
운영 확인	transactionId/operationId 로 ADM/owner query 확인

17.15 AI Optional

[PRODUCT CONTRACT]

항목	내용
구현 흐름	provider router + policy
개발 입력	업무 key/owner/profile/timeout/idempotency/security policy
정상 판정	API/DB/provider side effect 와 transaction lineage 가 일치
Fault	PII/model timeout/cost cap
복구	fallback/deny/approval
운영 확인	transactionId/operationId 로 ADM/owner query 확인

REFERENCE AI optional flow

● ● ● CODE · REFERENCE AI optional flow

```
AiRequest safe = policy.maskAndClassify(input, principal);
AiDecision route = router.select(safe.modelClass(), safe.dataClass(), safe.costBudget());
if (route.highRisk() && !approval.validFor(safe.requestHash())) throw new ApprovalRequired();
AiResult result = client.call(route.provider(), safe, deadline);
audit.recordModelUsage(result.model(), result.tokens(), result.cost(), safe.transactionId());
```

18. EDU 실행 절차 - 16 개 Transaction Lab 상세

18.1 LOCAL commit/rollback

Lab field	값
requirementId	EDU-TX-01
objective	LOCAL commit/rollback 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	./gradlew :cpf-reference:test --tests "*Local*Transaction*"
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	rollback 후 business row/outbox 변화 없음
failure scenario	DB 예외를 commit 직전에 주입
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-01"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-01-000001"
./gradlew :cpf-reference:test --tests "*Local*Transaction*"
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.2 XA DB+DB

Lab field	값
requirementId	EDU-TX-02
objective	XA DB+DB 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	transaction harness run xa-db-db
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	recovery 후 두 DB outcome 동일
failure scenario	resource2 prepare 이후 failure
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-02"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-02-000001"
transaction harness run xa-db-db
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.3 XA DB+JMS

Lab field	값
requirementId	EDU-TX-03
objective	XA DB+JMS 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	transaction harness run xa-db-jms
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	DB/JMS outcome 동일 또는 explicit in-doubt
failure scenario	JMS prepare/commit failure
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-03"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-03-000001"
transaction harness run xa-db-jms
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.4 XA prepare kill

Lab field	값
requirementId	EDU-TX-04
objective	XA prepare kill 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	transaction harness run xa-kill-after-prepare
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	restart scan 후 in-doubt 0
failure scenario	prepare 직후 process kill
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-04"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-04-000001"
transaction harness run xa-kill-after-prepare
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.5 Outbox

Lab field	값
requirementId	EDU-TX-05
objective	Outbox 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	reference event outbox
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	business committed + outbox UNKNOWN→CONFIRMED
failure scenario	Broker ACK loss
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-05"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-05-000001"
reference event outbox
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.6 Publisher kill

Lab field	값
requirementId	EDU-TX-06
objective	Publisher kill 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	outbox publisher process
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	lease expiry 후 new fence reclaim
failure scenario	CLAIMED 직후 kill
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-06"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-06-000001"
outbox publisher process
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.7 Inbox dedup

Lab field	값
requirementId	EDU-TX-07
objective	Inbox dedup 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	consumer duplicate test
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	업무 side effect exactly once
failure scenario	same message concurrent 20 calls
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-07"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-07-000001"
consumer duplicate test
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.8 Saga success/fail

Lab field	값
requirementId	EDU-TX-08
objective	Saga success/fail 의 정상 · 실패 · 복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	saga A-B-C
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	B/A compensation reverse order
failure scenario	C failure
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txId/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-08"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-08-000001"
saga A-B-C
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.9 Saga compensation fail

Lab field	값
requirementId	EDU-TX-09
objective	Saga compensation fail 의 정상 · 실패 · 복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	saga compensation
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	UNKNOWN→retry/manual review
failure scenario	B compensation timeout
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-09"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-09-000001"
saga compensation
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.10 TCC confirm

Lab field	값
requirementId	EDU-TX-10
objective	TCC confirm 의 정상 · 실패 · 복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	tcc try-confirm
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	same terminal result replay
failure scenario	duplicate confirm
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-10"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-10-000001"
tcc try-confirm
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.11 TCC cancel

Lab field	값
requirementId	EDU-TX-11
objective	TCC cancel 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	tcc try-cancel
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	CANCELED or explicit UNKNOWN
failure scenario	empty rollback + hanging
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-11"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-11-000001"
tcc try-cancel
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.12 External timeout

Lab field	값
requirementId	EDU-TX-12
objective	External timeout 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	online A-B-C
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	UNKNOWN→D reconcile
failure scenario	remote side effect then response loss
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-12"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-12-000001"
online A-B-C
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.13 Local/Remote parity

Lab field	값
requirementId	EDU-TX-13
objective	Local/Remote parity 의 정상 · 실패 · 복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	same facade contract
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	response/error/idempotency same
failure scenario	switch adapter topology
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-13"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-13-000001"
same facade contract
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.14 Batch lineage

Lab field	값
requirementId	EDU-TX-14
objective	Batch lineage 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	batch A-B-C
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	same txId + new execution/attempt linked
failure scenario	kill after checkpoint
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txId/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-14"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-14-000001"
batch A-B-C
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.15 ADM Timeline

Lab field	값
requirementId	EDU-TX-15
objective	ADM Timeline 의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	integrated trace
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	PARTIAL/STALE explicit
failure scenario	one source temporarily unavailable
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-15"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-15-000001"
integrated trace
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

18.16 Starter DX

Lab field	값
requirementId	EDU-TX-16
objective	Starter DX의 정상·실패·복구 계약을 실행으로 확인
prerequisite	clean test data, required DB/Broker/provider, isolated service identity
source location	cpf-reference / official public contract consumer
command	generated domain profile
input	고정 requestId/idempotencyKey/transactionId + deterministic test data
expected normal	fail-fast message + no silent fallback
failure scenario	remove required provider config
recovery	결과 조회→정책에 맞는 restart/retry/reconcile/compensation
cleanup	test DB/message/temp/lease 를 owner cleanup 으로 제거
evidence	command/exit code/state before-after/txid/log or report hash

PowerShell/Bash 개념 예

● ● ● TERMINAL · 실행 기록 예

```
# 실행 기록 예
$env:CPF_EDU_SCENARIO="EDU-TX-16"
$env:CPF_TEST_TRANSACTION_ID="TX-EDU-16-000001"
generated domain profile
# 실패 주입 후 동일 식별자로 상태 조회 / 복구 수행
# 정상화 판정 결과와 transactionId 를 evidence 에 기록
```

19. 현행 Public API·Starter 사용 진입점

최신 Source 에서 직접 확인한 개발 진입점이다. 아래 이름은 실제 Symbol/경로이며, 각 업무 Domain 은 필요한 Capability 만 선택해 Consumer 로 사용한다.

영역	Symbol	Source	사용 경계
Transaction	CpfTransactionStrategy	cpf-core/.../api/transaction/ CpfTransactionStrategy.java	LOCAL/XA_JTA/OUTBOX/ INBOX_DEDUP/SAGA/TCC 선택
Transaction	CpfTransactionDefinition	cpf-core/.../api/transaction/ CpfTransactionDefinition.java	strategy/propagation/ isolation/timeout/readOnly; localDefault() 제공
Transaction	CpfTransactionOperations	cpf-core/.../api/transaction/ CpfTransactionOperations.java	execute(definition, work)
TCC	CpfTccParticipant / CpfTccRecoveryOperations	cpf-core/.../api/transaction	Try/Confirm/Cancel 및 recovery contract
XA	CpfXaTransactionManager / CpfXaRecoveryRecord	cpf-core/.../api/transaction	XA transaction/recovery provider 경계
Persistence	CpfCrudRepositoryPort	cpf-core/.../api/persistence	create/find/update/delete/ exists/count + native escape hatch
Persistence	CpfSearchRepositoryPort / CpfSearchSpec	cpf-core/.../api/persistence	filter/search contract
Persistence	CpfPersistencePolicy	cpf-core/.../api/persistence/ CpfPersistencePolicy.java	sort/filter/bulk/page validation
JPA	CpfJpaOperations	cpf-starters/data/persistence- jpa	CRUD · bulk · lock · timeout · exc eption · query observation
AI	CpfAiOperations	cpf-core/.../api/ai/ CpfAiOperations.java	execute(CpfAiRequest)
Signature	CpfDigitalSignatureOperations	cpf-core/.../api/security/ crypto	sign/ verify(transactionId, keyId, alg orithm, payload)
OIDC	CpfOidcProperties / PrincipalMapper	cpf-starters/security/oidc- login	claim/role/scope mapping
SOAP	CpfSoapClient / CpfSoapProperties	cpf-starters/integration/soap	connect/read timeout 기반 SOAP client
Audit	CpfTamperAuditOperations / Store	cpf-core/.../api/security/audit	hash-chain audit contract
Reference	ReferenceDurableTccReservationParticipant	cpf-reference/.../transaction/ tcc	TCC durable state/recovery 예 제
Reference	ReferenceAiEducationController	cpf-reference/.../online/ai	AI Public API Consumer 예제
Reference	ReferenceSoapEducationController	cpf-reference/.../online/soap	SOAP Starter Consumer 예제
Harness	CpfXaCrashRecoveryHarness	cpf-starters/data/transaction- jta/src/test	prepare→kill→restart recovery 검증용 Harness

19.1 Transaction 호출 패턴

현재 Public API 의 핵심 호출 형태는 `CpfTransactionOperations.execute(CpfTransactionDefinition, CpfTransactionWork)` 다. `CpfTransactionWork` 는 `CpfTransactionContext` 를 인자로 받으며, LOCAL 기본 정의는 `CpfTransactionDefinition.localDefault()` 에서 생성한다.

XA/JTA 는 `cpf.data.transaction.jta.enabled` 가 기본 false 다. LOCAL 사용자가 XA Provider 를 설치·설정하지 않아도 되는 경계를 유지한다.

19.2 JPA·AI·SOAP·OIDC 설정 경계

Prefix	확인된 기본값/경계	변경 특성
cpf.data.persistence.jpa	queryTimeoutMs=3000, slowQueryThresholdMs=500, bulkFlushSize=100, maxBulkSize=1000, requireJta=false	RESTART_REQUIRED
cpf.data.transaction.jta	enabled=false, standalone=false, defaultTimeoutSeconds=30, startupRecovery=true	JTA 선택 시만 활성화
cpf.integration.ai	enabled=false, maxAttempts=3, retryAttemptsPerProvider=2, timeout=10s, circuit threshold/open duration	RESTART_REQUIRED
cpf.integration.soap	connectTimeout=2s, readTimeout=5s	RESTART_REQUIRED
cpf.security.oidc	enabled=false, userIdClaim=sub, tenantClaim=tenant_id, role/scope mapping, safeClaimNames	RESTART_REQUIRED / secretSeparated

21. 실행형 개발 워크북 - 복사해서 시작하는 기준

문서 사용 규칙

이 장은 “설명만 읽는 절”이 아닙니다. 파일 경로 → Source → Config/SQL → 실행 명령 → 기대 결과 → Fault Injection → 복구/Test 순서로 닫습니다. CURRENT SOURCE 는 07_15 구현, 07_16 신규 Capability 는 PRODUCT CONTRACT/REFERENCE 로 명확히 구분합니다.

21.1 Generator 에서 첫 실행까지

● ● ● TERMINAL · create-domain.ps1

```
pwsh -NoProfile -File .\cpf-tools\generator\create-domain.ps1 -DomainName Payment -SystemCode PAY -Package com.customer.payment -DatabaseVendor postgresql -Profile secure-api  
.\gradlew.bat :cpf-payment:clean :cpf-payment:test
```

● ● ● CODE · REFERENCE

```
cpf-payment/  
├─ build.gradle  
├─ src/main/java/com/customer/payment/  
│ └─ api/  
│ └─ application/  
│ └─ domain/  
│ └─ port/  
│ └─ adapter/  
│ └─ config/  
├─ src/main/resources/db/migration/  
└─ src/test/java/com/customer/payment/
```

● ● ● OUTPUT · Generator/Build

```
DomainName=Payment  
SystemCode=PAY  
Profile=secure-api  
resolvedStarters=[...]  
BUILD SUCCESSFUL
```

21.2 업무 API 전체 예제 - Transfer

아래 예제는 PRODUCT CONTRACT 를 구현하는 REFERENCE 입니다. CPF 내부 구현 Symbol 이라고 주장하지 않으며 고객 Domain 에서 복사·변형할 수 있는 완성 단위입니다.

● ● ● SOURCE · src/main/java/com/customer/payment/transfer/api/TransferRequest.java

```
package com.customer.payment.transfer.api;

import java.math.BigDecimal;
import java.util.Objects;

public record TransferRequest(
    String requestId,
    String idempotencyKey,
    String debitAccountId,
    String creditAccountId,
    BigDecimal amount,
    String currency) {
    public TransferRequest {
        if (requestId == null || requestId.isBlank()) throw new IllegalArgumentException("requestId");
        if (idempotencyKey == null || idempotencyKey.isBlank()) throw new IllegalArgumentException("idempotencyKey");
        if (debitAccountId == null || debitAccountId.isBlank()) throw new IllegalArgumentException("debitAccountId");
        if (creditAccountId == null || creditAccountId.isBlank()) throw new IllegalArgumentException("creditAccountId");
        if (Objects.equals(debitAccountId, creditAccountId)) throw new IllegalArgumentException("same account");
        if (amount == null || amount.signum() <= 0) throw new IllegalArgumentException("amount");
        if (currency == null || !currency.matches("[A-Z]{3}")) throw new IllegalArgumentException("currency");
    }
}
```

● ● ● SOURCE · TransferController.java

```
@RestController
@RequestMapping("/api/transfers")
public class TransferController {
    private final TransferFacade facade;

    public TransferController(TransferFacade facade) { this.facade = facade; }

    @PostMapping
    ResponseEntity<TransferResult> create(
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @RequestBody TransferRequest body) {
        TransferRequest request = new TransferRequest(
            body.requestId(), idempotencyKey, body.debitAccountId(),
            body.creditAccountId(), body.amount(), body.currency());
        TransferResult result = facade.transfer(request);
        return ResponseEntity.accepted().body(result);
    }
}
```

● ● ● SOURCE · TransferService.java

● ● ● SOURCE · src/main/java/com/customer/payment/transfer/api/TransferRequest.java

```
public final class TransferService implements TransferFacade {
    private final TransferRepository repository;
    private final AccountPort accountPort;
    private final TransferEventPort eventPort;

    @Override
    public TransferResult transfer(TransferRequest r) {
        var replay = repository.findByIdempotencyKey(r.idempotencyKey());
        if (replay.isPresent()) return replay.get().toResult();

        var transfer = Transfer.pending(r);
        repository.insertPending(transfer);
        try {
            var external = accountPort.transfer(transfer.toAccountCommand());
            transfer.markExternalAccepted(external.providerRequestId());
            repository.commitAccepted(transfer);
            eventPort.enqueueCompleted(transfer.toEvent());
            return transfer.toResult();
        } catch (RemoteResultUnknownException unknown) {
            transfer.markUnknown(unknown.providerRequestId());
            repository.markUnknown(transfer);
            return transfer.toResult();
        } catch (RuntimeException failure) {
            repository.rollbackPending(transfer.id());
            throw failure;
        }
    }
}
```

● ● ● SQL · V001__create_pay_transfer.sql

```
CREATE TABLE pay_transfer (
    transfer_id    VARCHAR(64) NOT NULL,
    idempotency_key VARCHAR(128) NOT NULL,
    request_hash   CHAR(64) NOT NULL,
    state          VARCHAR(24) NOT NULL,
    provider_request_id VARCHAR(128),
    version_no     BIGINT NOT NULL DEFAULT 0,
    created_at     TIMESTAMP NOT NULL,
    updated_at     TIMESTAMP NOT NULL,
    PRIMARY KEY (transfer_id),
    CONSTRAINT uq_pay_transfer_idem UNIQUE (idempotency_key)
);
CREATE INDEX ix_pay_transfer_state_time ON pay_transfer(state, updated_at);
```

● ● ● CONFIG · application-payment.yml

```
cpf:
  payment:
    transfer:
      remote-timeout: 3s
      idempotency-ttl: 24h
      reconcile-page-size: 100
```

● ● ● OUTPUT · 정상 요청

```
POST /api/transfers HTTP/1.1
Content-Type: application/json
Idempotency-Key: pay-20260809-000001
```

```
{"requestId":"REQ-000001","debitAccountId":"A1001","creditAccountId":"A2001","amount":125000.00,"currency":"KRW"}
```

● ● ● SOURCE · src/main/java/com/customer/payment/transfer/api/TransferRequest.java

● ● ● OUTPUT · 정상

```
HTTP/1.1 202 Accepted
{"transferId":"TR-01J...", "state":"ACCEPTED", "transactionId":"<trusted transactionId>"}
```

● ● ● HTTP · idempotency conflict

```
HTTP/1.1 409 Conflict
{"code":"PAY-TRANSFER-IDEMPOTENCY-CONFLICT", "retryable":false, "transactionId":"..."}
```

21.3 CURRENT SOURCE - Transaction Contract

CURRENT SOURCE · `b2da6bd720d1`에서 확인한 실제 계약입니다.

● ● ● SOURCE · cpf-core/.../CpfTransactionStrategy.java

```
public enum CpfTransactionStrategy {
    LOCAL,
    XA_JTA,
    OUTBOX,
    INBOX_DEDUP,
    SAGA,
    TCC
}
```

● ● ● SOURCE · cpf-core/.../CpfTransactionDefinition.java

```
public record CpfTransactionDefinition(
    CpfTransactionStrategy strategy,
    CpfTransactionPropagation propagation,
    CpfTransactionIsolation isolation,
    Duration timeout,
    boolean readOnly) {
    public static CpfTransactionDefinition localDefault() {
        return new CpfTransactionDefinition(
            CpfTransactionStrategy.LOCAL,
            CpfTransactionPropagation.REQUIRED,
            CpfTransactionIsolation.DEFAULT,
            Duration.ofSeconds(30), false);
    }
}
```

● ● ● SOURCE · cpf-core/.../CpfTransactionOperations.java

```
public interface CpfTransactionOperations {
    <T> T execute(CpfTransactionDefinition definition, CpfTransactionWork<T> work);
}
```

21.4 CURRENT SOURCE - Online A→B→C(/D)와 UNKNOWN

CURRENT SOURCE ·

`cpf-reference/src/main/java/com/cpf/reference/online/integrated/OnlineAbcdReferenceFlow.java`는 Controller → Domain A → B → C(/D) → JDBC/Remote를 연결하고 remote side effect 이후 local DB 실패를 UNKNOWN으로 분류합니다.

● ● ● SOURCE · OnlineAbcdReferenceFlow.DomainB

```
public Result execute(Request r) {
    List<Event> e = new ArrayList<>();
    try {
        repo.begin(r.businessKey());
        String remote = c.execute(r, e);
        if (failAfterRemote) throw new LocalDbFailure("after-remote");
        repo.save(r.businessKey(), remote);
        repo.commit(r.businessKey());
        return new Result(r.transactionId(), r.businessKey(), r.attempt(), Outcome.SUCCESS, List.copyOf(e));
    } catch (LocalDbFailure ex) {
        repo.rollback(r.businessKey());
        boolean remoteSucceeded = e.stream().anyMatch(x -> x.segment().equals("C") && x.state().equals("REMOTE_SUCCESS"));
        return new Result(r.transactionId(), r.businessKey(), r.attempt(),
            remoteSucceeded ? Outcome.UNKNOWN : Outcome.FAILED, List.copyOf(e));
    }
}
```

● ● ● TERMINAL · Reference Test 실행

```
./gradlew :cpf-reference:test --tests "*OnlineAbcd*"
```

● ● ● CODE · response loss/local DB failure

```
transactionId=<same id>
C.REMOTE_SUCCESS
B.UNKNOWN
D.RECONCILED # compensation/reconcile 가 성공했을 때
```

21.5 CURRENT SOURCE - Spring Data JPA Starter

● ● ● CODE · CpfJpaProperties defaults

```
cpf.data.persistence.jpa.query-timeout-ms=3000
cpf.data.persistence.jpa.slow-query-threshold-ms=500
cpf.data.persistence.jpa.bulk-flush-size=100
cpf.data.persistence.jpa.max-bulk-size=1000
cpf.data.persistence.jpa.require-jta=false
```

● ● ● SOURCE · CpfJpaOperations 핵심 사용면

```
public <T> T create(T value) { entityManager.persist(value); return value; }
public <T> T update(T value) { return entityManager.merge(value); }
public <T, ID> Optional<T> findById(Class<T> type, ID id) {
    return Optional.ofNullable(entityManager.find(type, id));
}
public <T, ID> Optional<T> findById(Class<T> type, ID id, CpfLockMode lockMode) {
    return Optional.ofNullable(entityManager.find(type, id, CpfJpaLockModes.toJpa(lockMode)));
}
public int bulkPersist(Collection<?> values) {
    CpfPersistencePolicy.requireBulkSize(values.size(), properties.getMaxBulkSize());
    // flush/clear by bulkFlushSize
    ...
}
```

수정 경계

업무 개발자는 Entity/Repository/Specification/@Query 를 소유할 수 있지만 CPF 가 제공하는 max page/bulk, timeout, audit, transactionId, tenant/security 정책을 우회하지 않습니다. Native EntityManager escape 는 고급 SQL/Hint 가 필요한 경우에만 사용합니다.

21.6 CURRENT SOURCE - JTA/XA Starter ✎ Recovery

● ● ● CONFIG · `cpf.data.transaction.jta`

```
cpf:
  data:
    transaction:
      jta:
        enabled: true
        standalone: false
        default-timeout-seconds: 30
        startup-recovery: true
```

● ● ● TERMINAL · `run-xa-crash-recovery.ps1`

```
pwsh -NoProfile -File .\cpf-tools\verification\run-xa-crash-recovery.ps1
```

● ● ● CODE · EXPECTED RECOVERY CHECKPOINTS

```
PREPARE_REACHED
PROCESS_KILLED
TRANSACTION_MANAGER_RESTARTED
IN_DOUBT_SCANNED
RECOVERY_TERMINAL_STATE=<COMMITTED|ROLLED_BACK|MANUAL_REVIEW>
DUPLICATE_SIDE_EFFECT=0
```

21.7 CURRENT SOURCE - AI / SOAP / OIDC

● ● ● SOURCE · `CpfAiOperations`

```
public interface CpfAiOperations {
    CpfAiResponse execute(CpfAiRequest request);
}
```

● ● ● CONFIG · `cpf.integration.ai`

```
cpf:
  integration:
    ai:
      enabled: false
      provider-order: []
      max-attempts: 3
      retry-attempts-per-provider: 2
      timeout: 10s
      retry-backoff: 100ms
      circuit-failure-threshold: 3
      circuit-open-duration: 30s
```

● ● ● CONFIG · `cpf.integration.soap`

```
cpf:
  integration:
    soap:
      connect-timeout: 2s
      read-timeout: 5s
```

● ● ● CONFIG · `cpf.security.oidc`

● ● ● SOURCE · CpfAiOperations

```
cpf:
  security:
    oidc:
      enabled: true
      user-id-claim: sub
      tenant-claim: tenant_id
      group-claim: groups
      role-claim: realm_access.roles
      scope-claim: scope
      role-prefix: ROLE_
      scope-prefix: SCOPE_
```

21.8 07_16 신규 Capability - PRODUCT CONTRACT/REFERENCE

다음 6 개는 Canonical Requirement 에 추가됐지만 `07\_16` 자체에는 Product Source 추가가 없습니다. 따라서 아래 코드는 REFERENCE 이며 CURRENT SOURCE 로 표기하지 않습니다.

● ● ● CODE · Foundation convenience 사용

```
Clock clock = CpfFoundation.clock();
BigDecimal amount = CpfDecimals.money("125000.00", Currency.getInstance("KRW"));
String requestId = CpfIds.newRequestId();
CpfValidation.requireNotBlank("accountId", accountId);
// Core Utility 를 직접 늘리지 않고 owner capability/foundation 으로 분리
```

● ● ● CONFIG · Distributed Session Valkey

```
cpf:
  security:
    session:
      provider: valkey
      ttl: 30m
      rotation-on-authentication: true
      concurrent-session-limit: 3
      fail-closed-on-provider-unknown: true
```

● ● ● CODE · Object Storage Provider-neutral API

```
ObjectUpload upload = objectStorage.beginUpload(
  ObjectUploadRequest.of(tenantId, objectKey, contentType, expectedChecksum));
upload.write(stream);
ObjectMetadata committed = upload.commit();
// timeout/partial failure -> uploadId + objectKey + checksum 으로 reconcile
```

● ● ● CODE · Event Schema governance

```
event:
  name: PaymentCompleted
  version: 3
  contentType: application/avro
  compatibility: BACKWARD
  breakingChangeGate: true
```

● ● ● SOURCE · GraphQL Resolver 는 Service 를 재사용

● ● ● CODE · Foundation convenience 사용

```
@Controller
class PaymentQueryController {
    private final PaymentQueryService service;
    @RequestMapping
    PaymentView payment(@Argument String id) {
        return service.find(id); // REST Controller 와 동일 Application Service
    }
}
```

● ● ● HTTP · SSE consumer

```
GET /api/operations/{operationId}/events HTTP/1.1
Accept: text/event-stream
Last-Event-ID: 1842
Authorization: Bearer <token>
```

● ● ● CODE · duplicate/reconnect safe

```
id: 1843
event: progress
data: {"operationId":"OP-1","state":"RUNNING","progress":74}
```

reconnect 시 Last-Event-ID 를 기준으로 중복 이벤트를 client/store 에서 idempotent 처리

21.9 Fault Injection 과 복구 판정

Fault	주입 지점	금지	복구	정상화 판정
Remote timeout	side effect 전/후	blind retry	result query → idempotent retry/reconcile	provider+DB+event 일치
DB commit 후 response loss	client response	상위 rollback 으로 해결 가능	idempotency replay / status query	stored result 재사용
Broker ACK loss	publish 후 ACK	무조건 재발행	outbox/inbox dedup	중복 business effect 0
XA prepare 후 kill	prepare/commit 사이	manual row 수정	TM restart/recovery scan	in-doubt 0 또는 manual review
TCC confirm loss	confirm response	Try 재수행	idempotent confirm/status query	reservation terminal
Valkey session outage	session lookup	fail-open auth	fail-closed + recovery	stale session 0

● ● ● TERMINAL · 개발자 인수 전 한 번에 확인

```
.\gradlew.bat clean test qualityGate
npm --prefix cpf-admin/frontend ci
npm --prefix cpf-admin/frontend run typecheck
npm --prefix cpf-admin/frontend run test
```

● ● ● CODE · 문서와 Runtime 을 구분한 인수 기록

```
Source/API/SQL/Config review : PASS
Unit/Contract test : <actual result only>
DB3 runtime : UNVERIFIED unless executed
Browser E2E : UNVERIFIED unless executed
Multi-instance/fault : UNVERIFIED unless executed
```

문서 기준 및 검증 범위

항목	기준
Repository / Branch	freeangelsun/202412_01_CPF / master
Work start / latest reviewed HEAD	64fd08d963927860e8d023403dfa276931801ee5 (07_17)
Latest product source change basis	4c4248a12e699c07f9f5fb11fbb33b97ca04077d (07_16 canonical currentization source)
Canonical Requirement	186 개
CURRENT SOURCE	실제 Repository 에서 확인한 Symbol/경로
PRODUCT CONTRACT	현재 구현량과 분리한 제품 목표 계약
REFERENCE	복사·응용 가능한 완성 기준 예시; CURRENT SOURCE 아님
Runtime qualification	문서 세션에서 직접 실행하지 않은 DB/Browser/Multi-instance/Fault 는 미검증

22. 07_17 개발 Ownership 규칙

신규 Class/API/SPI 를 만들기 전에 “Core Kernel / Capability Contract / Provider / Starter / Customer Common / ADM / BZA / Batch / Gateway / 고객·기관 Adapter” 중 실제 Owner 를 먼저 결정합니다. interface·SPI·provider-neutral 이라는 형식만으로 Core 에 올리지 않습니다.

질문	YES 일 때	NO 일 때
대부분 Capability 가 공통으로 알아야 하는가?	계속 평가	해당 Owner/Capability
Optional 기능 없이도 필요한가?	계속 평가	Optional Capability
Runtime/Topology/Provider 독립인가?	계속 평가	Provider/Starter
기술 교체 후 의미가 유지되는가?	계속 평가	Capability Runtime
CPF 자체 전역 Semantics 인가?	Core 후보	Owner/Capability

● ● ● CODE · BAD · provider-neutral 이라 Core 에 배치

```
// BAD - 형식만 보고 Core 에 전용 capability API 를 적히
package com.cpf.core.api.ai;
public interface CpfAiOperations { /* ... */ }
```

● ● ● CODE · GOOD · Owner 별 Public API 사용

```
// GOOD - 전역 의미는 Core, 기능 전용 계약은 실제 Capability Owner
import com.cpf.core.api.transaction.CpfTransactionContext; // global semantics
// AI/File/Persistence/Batch/Gateway API 는 해당 Owner 공개 계약을 사용한다.
// exact package 는 CURRENT SOURCE relocation 이 반영된 뒤 현행화한다.
```

기능	개발 시 Owner 기준	주의
Error/Transaction/Context	cpf-core	전역 semantics 만
Paging/Persistence	data capability/provider	복잡 Query 는 Domain Port/native escape hatch
Header/Web	web/channel capability	Core 편의 API 로 끌어올리지 않음
File/AI/Notification/Webhook	해당 Optional Capability	미사용 시 0-footprint
Batch/Gateway	cpf-batch / cpf-gateway	전용 API·DTO·Port 는 Owner 소유
Fixed-Length	fixedlength-core + fixedlength	기관별 Mapping/Adapter 는 고객·기관 Owner

● ● ● TREE · Fixed-Length PRODUCT CONTRACT

```
cpf-starters/integration/fixedlength-core
├── Engine / Layout / Field / Group / Parser / Writer / encoding / byte-length / validation extension
cpf-starters/integration/fixedlength
├── Spring Runtime / AutoConfiguration
cpf-core
├── generic Error / Validation / Masking / Transaction Context only
customer/institution adapter owner
├── institution layout / mapping / endpoint / adapter / error policy
```

[CURRENT SOURCE GAP] 07_16 기준 실제 Source 에 Core 아래 남아 있는 AI/Persistence/Transaction 전용 계약은 현행 위치로 문서화하되, 신규 코드의 Owner 선택 근거로 사용하지 않습니다. 07_17 목표 경계와 구현 위치가 일치할 때까지 “재확인 필요”로 취급합니다.